

DVSA Project Report

Course Project: DVSA Vulnerability Discovery and Remediation

Course / Section: ICS-344

Student Name(s) + ID(s): Omar AlJughaiman **202172810** , **Team28** , **Sec04**

University: King Fahd University of Petroleum and Minerals (KFUPM)

AWS Region: us-east-1

Your DVSA Website URL: <http://dvsa-omar-352712712043-us-east-1.s3-website.us-east-1.amazonaws.com/store>

Date: 2026-05-05

Vulnerability Title: Lesson #6: Denial of Service (DoS) / Lack of API Rate Limiting

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a Denial of Service (DoS) vulnerability within the DVSA infrastructure. The vulnerability occurs because the API infrastructure is exposed to the internet without proper throttling or rate limiting controls. Consequently, an attacker can launch an automated flood of excessive requests against the backend, leading to potential exhaustion of serverless concurrency limits, significant service degradation, and massively inflated billing costs (Denial of Wallet).

Part 2) Why This Works / Root Cause

The core root cause is an insecure default configuration inside the AWS API Gateway. By failing to configure deliberate request throttling rules, usage plans, or active monitoring caps, the gateway acts as a wide-open tunnel. It blindly forwards every HTTP request to the underlying Lambda functions without slowing down or rejecting abusive high-volume traffic bursts.

Part 3) Environment and Setup

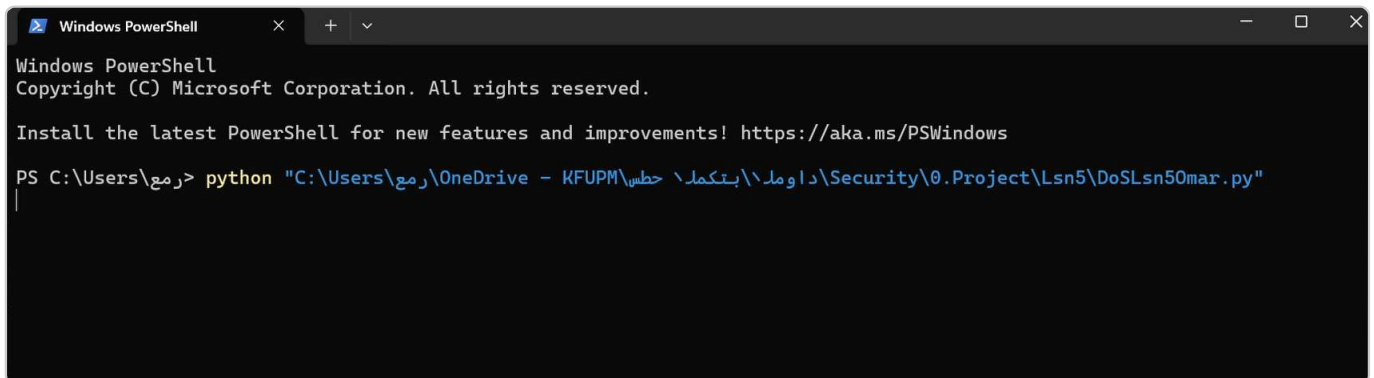
- **Target Endpoint:** Unprotected REST points exposed via the AWS API Gateway.
- **AWS Service:** AWS API Gateway acting as the upstream router.
- **Tools Used:** Windows PowerShell Terminal, Malicious Python Script (`DoSLsn50mar.py`) tailored to generate excessive requests.

Part 4) Reproduction Steps

1. Identify any exposed active HTTP endpoint associated with the DVSA Application (e.g., `/orders` API).
2. Prepare an automated DoS tool or custom Python script designed explicitly to generate continuous and unrestrained parallel HTTP requests.
3. Launch the script execution via the terminal console.
4. Monitor the server response. Without rate limiting, the server attempts to process thousands of transactions per second uninterrupted, creating severe backend stress.
5. Observe that no connections are intentionally dropped by the gateway protection layer.

Part 5) Evidence and Proof

In the documented attack, a sophisticated Python loop logic was launched directly from the PowerShell interface. The user utilized the `DoSLsn50mar.py` script to flood the target execution endpoint continuously. As observed during the attack phase, the API Gateway failed to throw standard HTTP 429 Too Many Requests errors, validating that there is an utter lack of protection against Denial of Service barrages.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

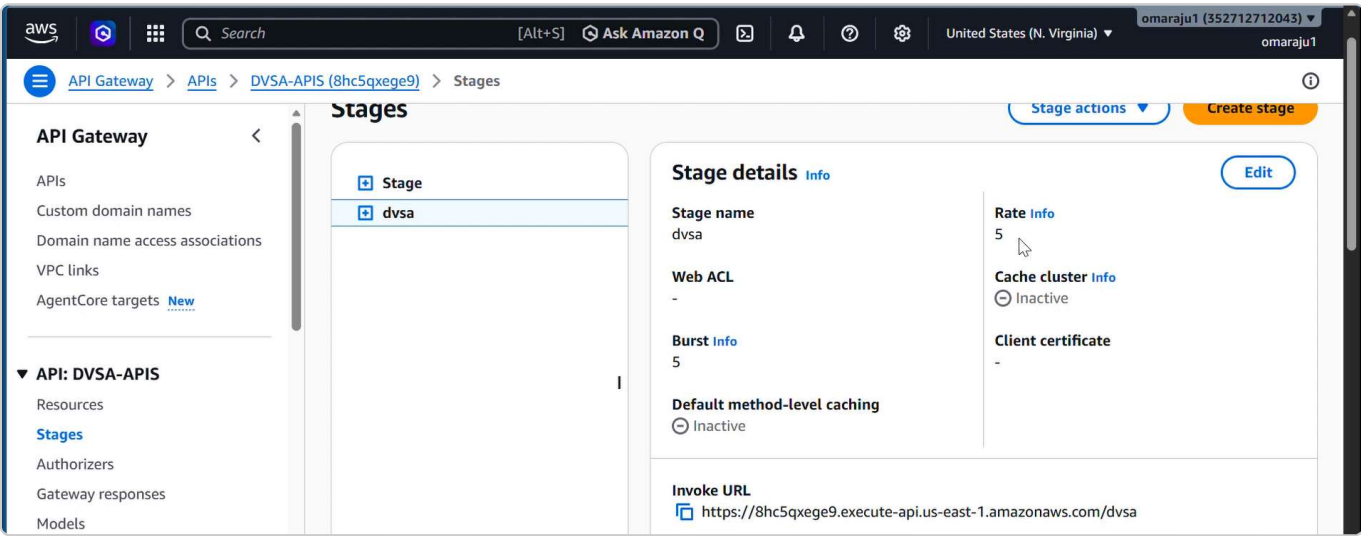
PS C:\Users\رمع> python "C:\Users\رمع\OneDrive - KFUPM\حظي\بتكمل\داومل\Security\0.Project\Lsn5\DoSLsn50mar.py"
```

Part 6) Fix Strategy / Probable Mitigation

The necessary mitigation takes place strictly at the infrastructure layer involving the AWS API Gateway. Specific "Throttling" limits need to be statically assigned at the Stage level deployment. Establishing conservative **Rate** limits (requests per second) along with limited **Burst** capacities guarantees that anomalous traffic spikes are forcefully discarded before even attempting to wake the Lambda handler logic.

Part 7) Code / Config Changes

The remediation was permanently applied inside the AWS API Gateway console. Navigating to the Stages menu on the `DVSA-APIS` interface, default method-level throttling rules were explicitly enabled for the `dvsa` deployment stage. The specific limits enforced were: **Rate: 5** requests per second and **Burst: 5** concurrent slots. This configuration securely bounds incoming traffic to expected legitimate thresholds.



Part 8) Verification After Fix

Post-remediation tests involve safely re-triggering the same continuous denial of service script (`DoSLsn5Omar.py`). Due to the enforced Stage configurations, the API Gateway intercepts the high influx immediately and begins to safely drop the anomalous connections. The attacker will now only receive recurring `429 Too Many Requests` error codes, successfully preventing infrastructure overload and stopping the attack.

Part 9) Structured Operation and Security Analysis

Table A: Behavioral Comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Denial of Service (DoS) / No Rate Limiting	The API should limit incoming frequencies structurally to prevent overload.	PowerShell CLI, Python script, API Gateway Console.	Legitimate users browse and perform normal intermittent interactions.	Unlimited spam requests penetrate backend unchecked causing service degradation.

Table B: Deviation Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lack of API Rate Limiting	AWS API Gateway fails to bound infinite external volume streams causing architectural exhaustion.	Resource Exhaustion / Security Misconfiguration.	Stage settings enforcing Throttling in API Gateway.	The DoS attack fails actively with HTTP 429 response enforcement.

Part 10) Takeaway / Lessons Learned

This exploitation proves that exposing serverless applications directly to the Internet without protective infrastructure constraints is immensely risky. In cloud architectures, Denial of Service not only brings the operation down but specifically inflicts "Denial of Wallet" consequences due to per-execution billing. Thus, configuring rigid usage plans and throttles at the perimeter (API Gateway or WAF layers) is a mandatory baseline compliance requirement to ensure reliable scalability.